

OpenSSH + FIDO workshop

Alan Alvarez & Dennis Hills | Yubico

Open Source Summit North America - May 2026

About this workshop

- Exercises:
 - <https://github.com/YubicoLabs/fido-openssh-workshop>
- Basic OpenSSH knowledge assumed
- Some demos require a GitHub or GitLab account
- If you need one: grab a FIDO Security Key!

Before we start...

- If you haven't already, download/install:
 - OpenSSH client (version 8.2+, ideally 8.9+)
\$ **ssh -V**
 - Python 3.9+
\$ **python3 -V**
 - Docker Desktop (or something similar)
<https://www.docker.com/products/docker-desktop/>
 - An Ubuntu image (latest)
\$ **docker pull ubuntu:latest**
 - FIDO command-line tools
<https://developers.yubico.com/libfido2/>
\$ **<pkg-mgr> install libfido2**
(fido2-tools on debian/ubuntu)

Overview

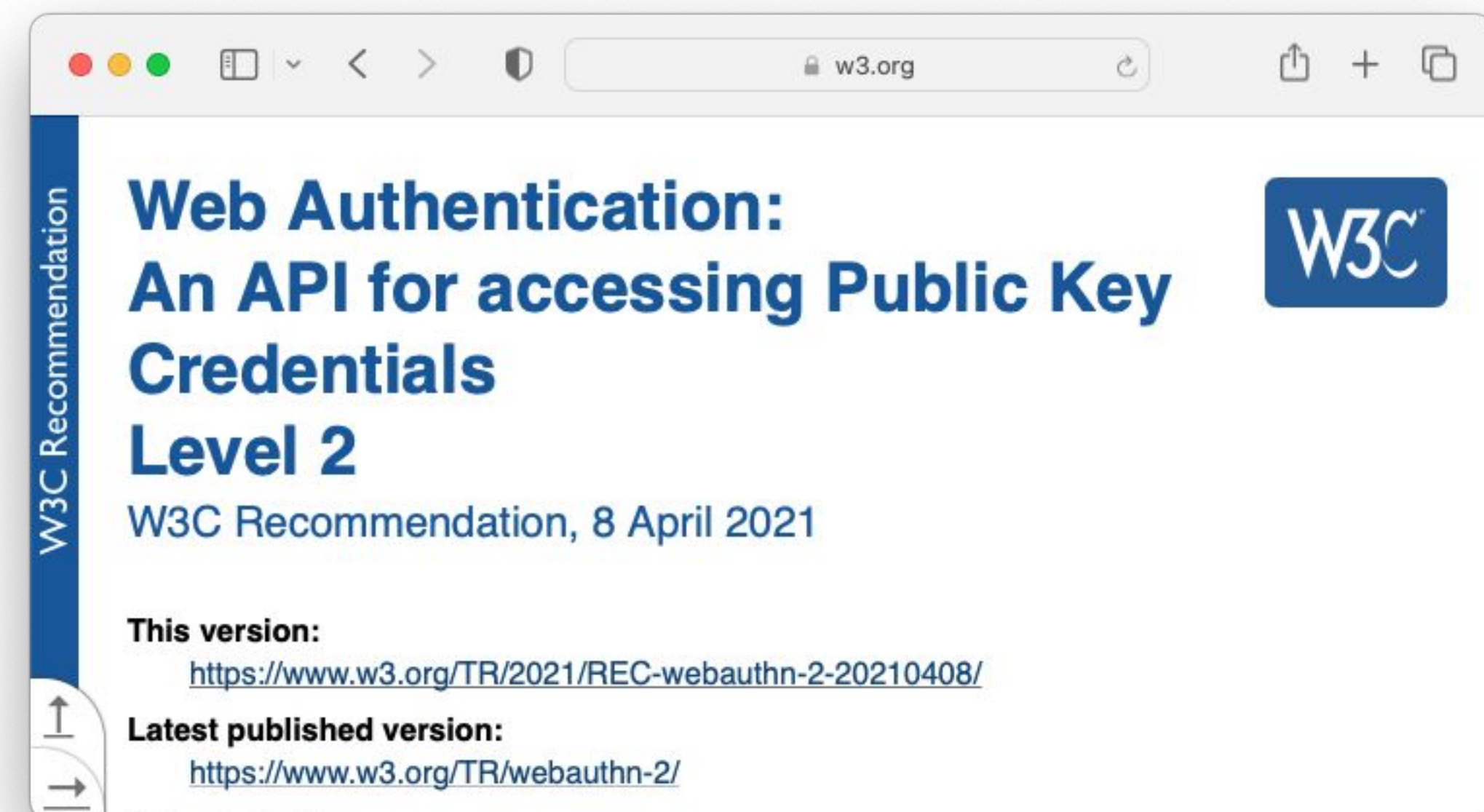
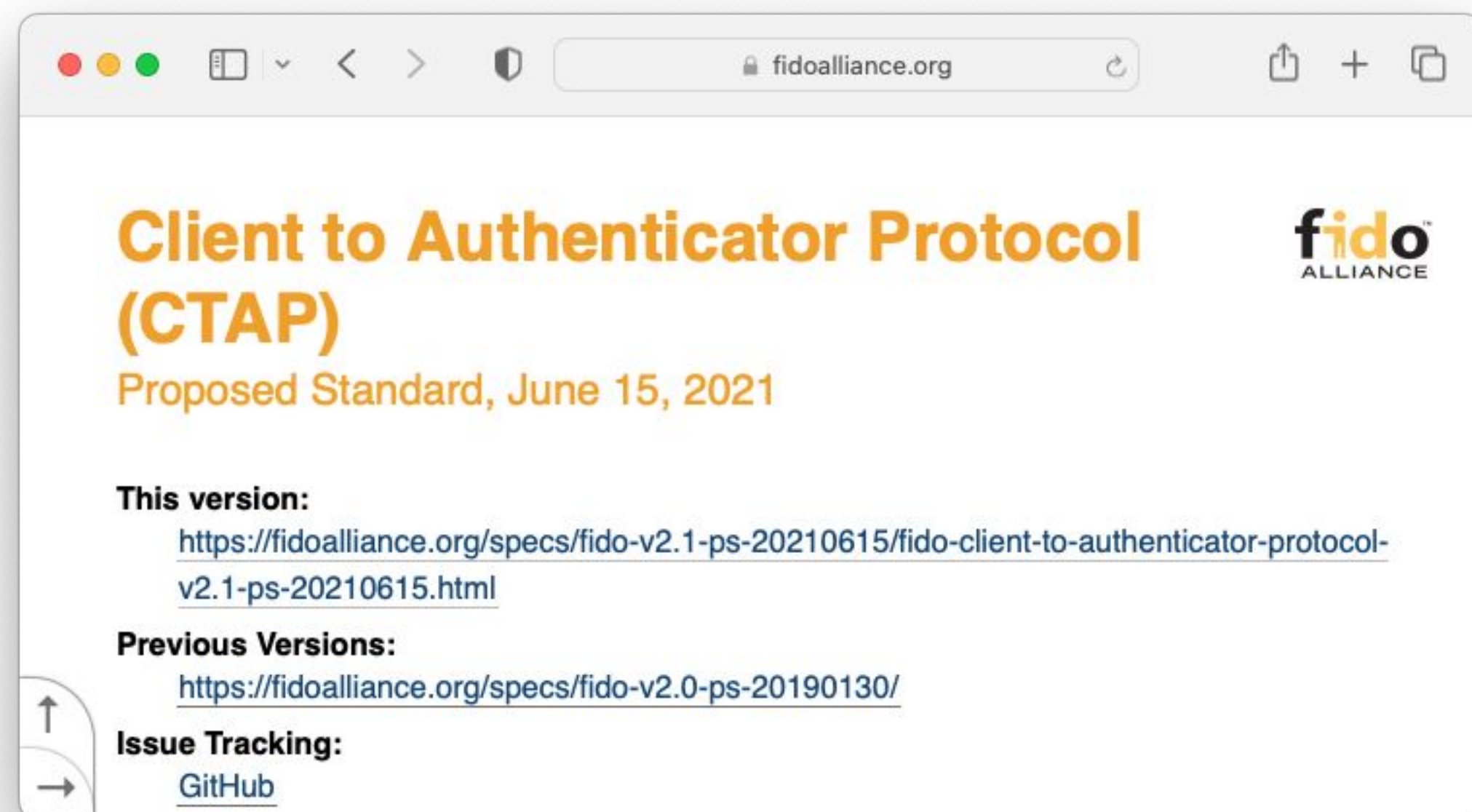
- Intro - what is FIDO?
 - Tools: libfido2
- Hardware-backed SSH private keys
 - Exercise: sign in using hardware-backed keys
- Generating SSH signatures using security keys
 - Exercise: git commit signing
- Storing SSH certificates
 - Exercise: largeBlobs
- Device attestation and the FIDO metadata service
 - Exercise: generating attestations
- ssh-agent with security keys
 - Exercise: more secure agent-forwarding

Introduction

FIDO2 - concepts

FIDO2

- Open standard for hardware based authentication
- Maintained by the FIDO Alliance: Google, Microsoft, Apple, Yubico
- The technology behind security keys and passkeys



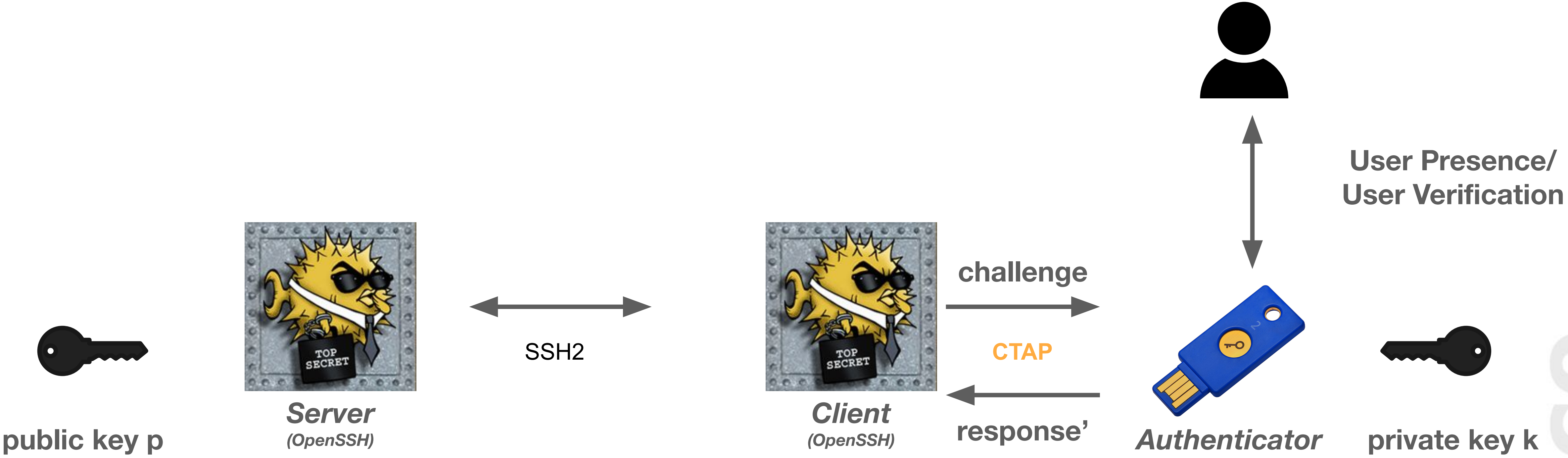
Why OpenSSH + FIDO?

- Your SSH private key lives on the hardware key, never on your machine
 - Can't be exported, can't be stolen
- Every operation requires physical interaction
 - Button touch = **User Present**
 - PIN or biometric = **User Verified**
- Sign git commits with your key
- Store SSH certificates on the key
- Prove which key generated your credential (**attestation**)
- Simpler than **OpenPGP** or **PKCS#11**

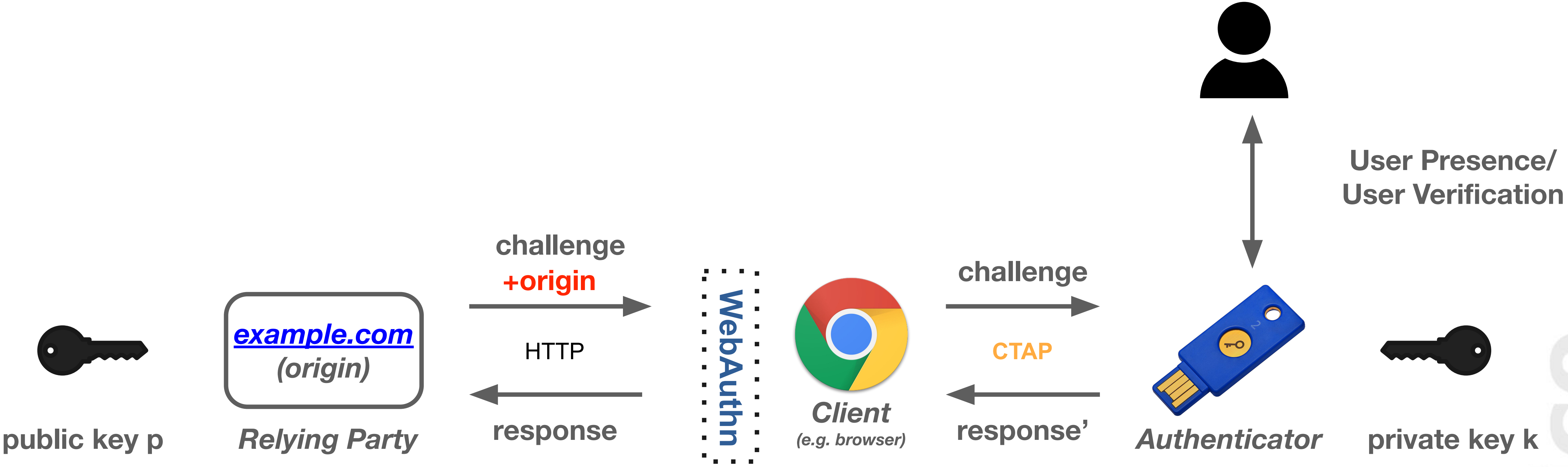


yubico

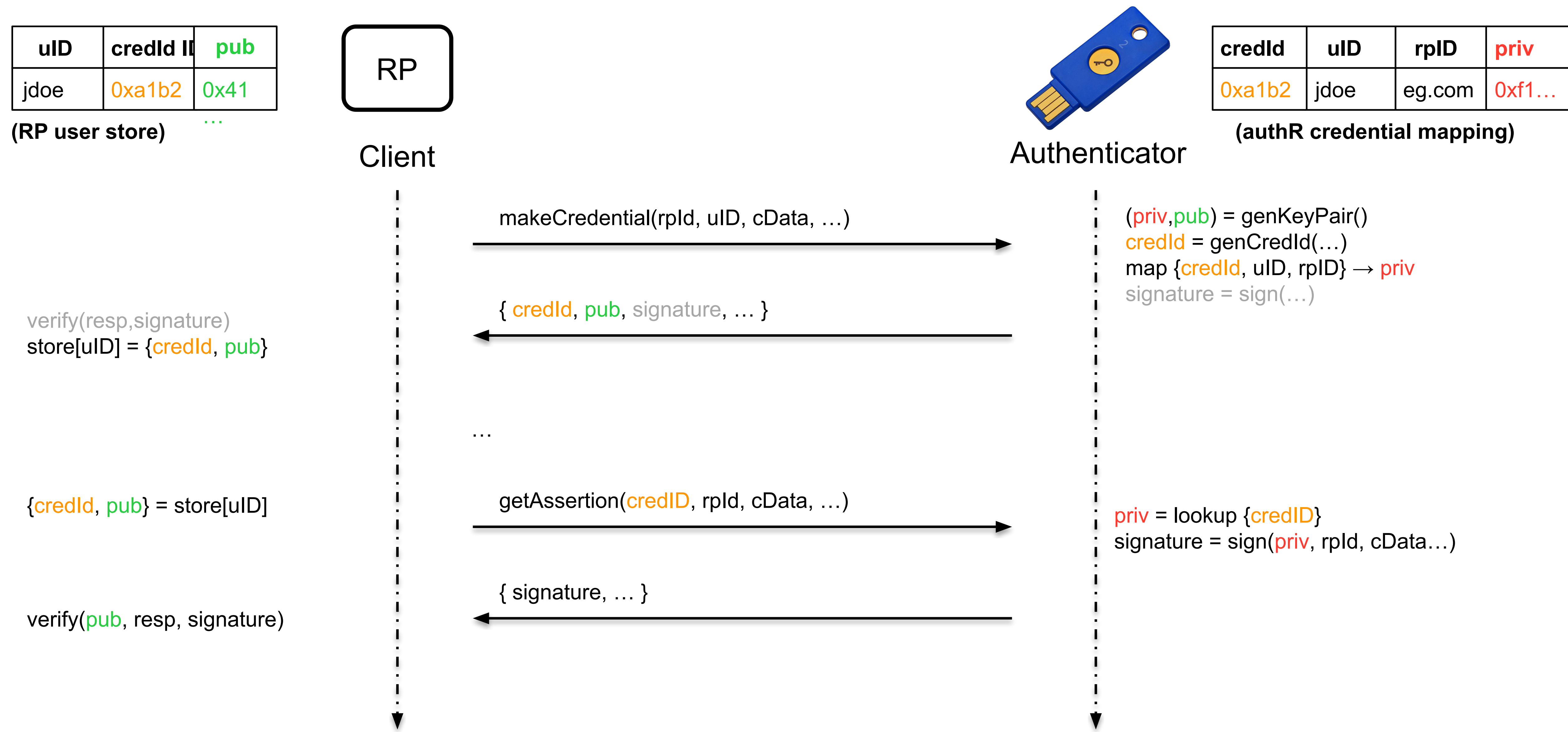
CTAP and SSH



CTAP and Webauthn (simplified)



CTAP: operations (simplified)



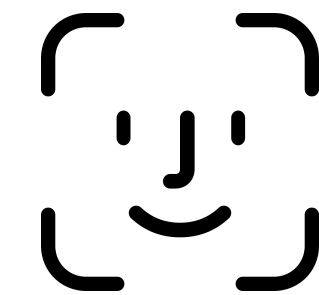
Authenticators

- ***cross-platform Authenticator***
also called ***roaming*** authenticator
example: a USB security key



- Roaming authenticators can use different ***transports***: USB, NFC, BLE, hybrid

- ***Platform Authenticator***
Built into user's device
example: a built-in fingerprint sensor



FaceID



TouchID



Windows
Hello

- Note: a single authenticator can store multiple FIDO credentials!

FIDO Security Keys: cross-platform authenticators



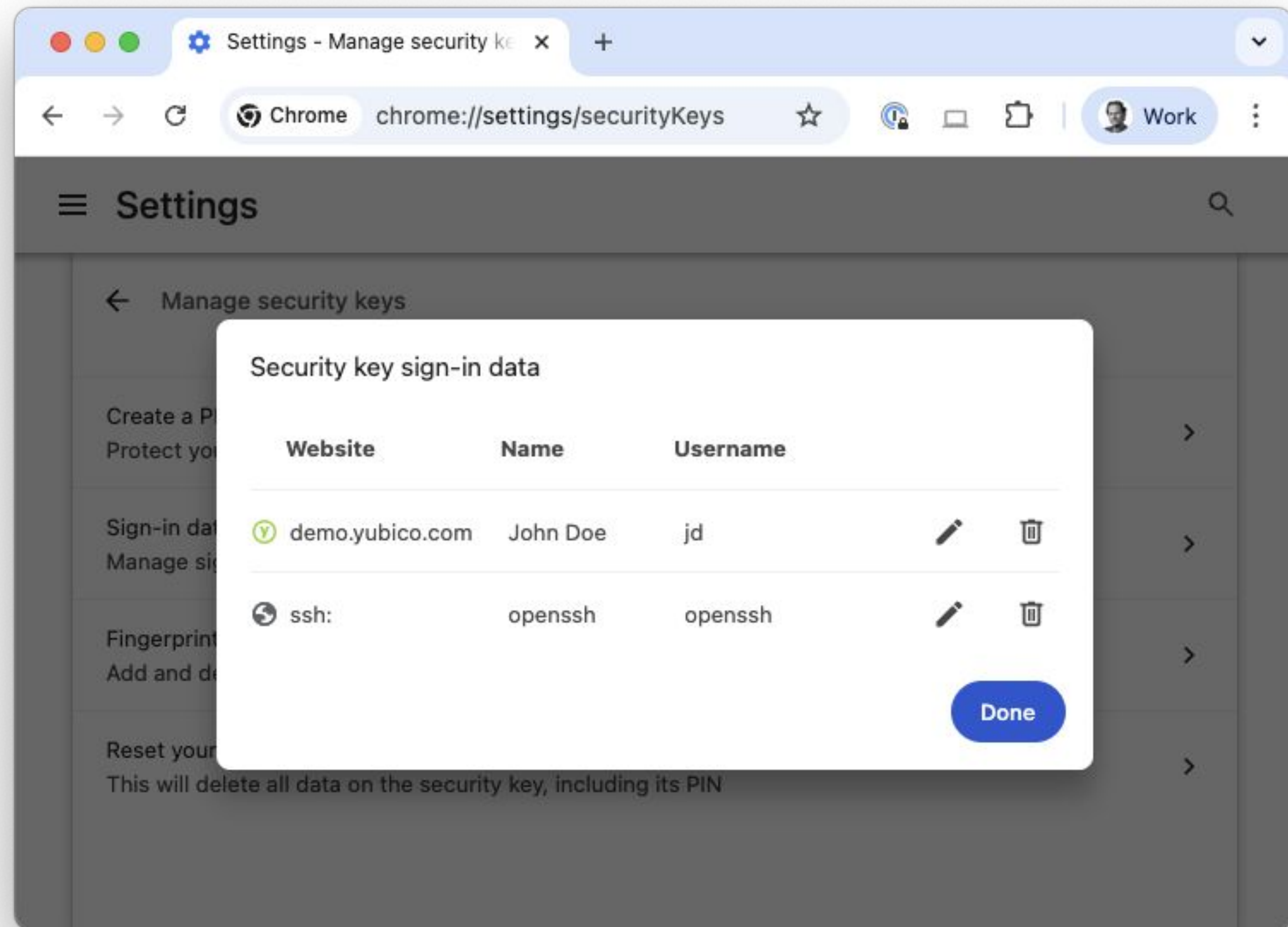
yubico

User Present vs User Verified

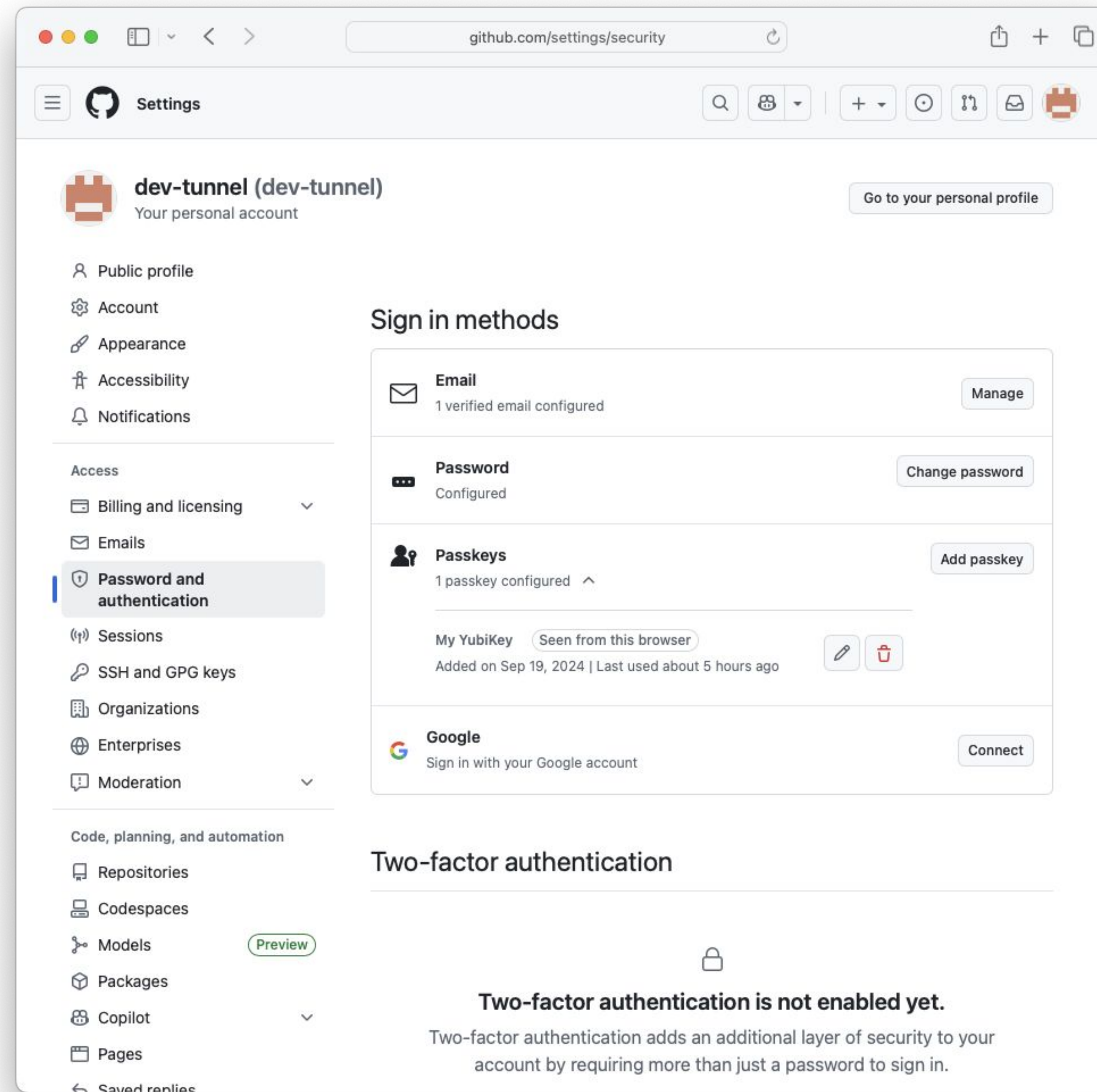
- User Present (UP): User is physically present
 - On a security key: a button touch
- User Verified (UV): User is verified, i.e. user authenticated to the Security Key
 - On a security key: entering a PIN
(on biometric models: touching a built-in fingerprint scanner)
 - On a platform authenticator: typically a biometric
(eg Touch ID/Face ID)

Resident vs non-resident Keys

- A Credential can be:
 - **Resident**: stored in a security key's internal memory
 - **Non-resident**: encrypted with a wrapping key and stored off-key
- Resident keys are called ***Discoverable Credentials*** since CTAP v2.1
- A security key can store a limited amount of resident keys



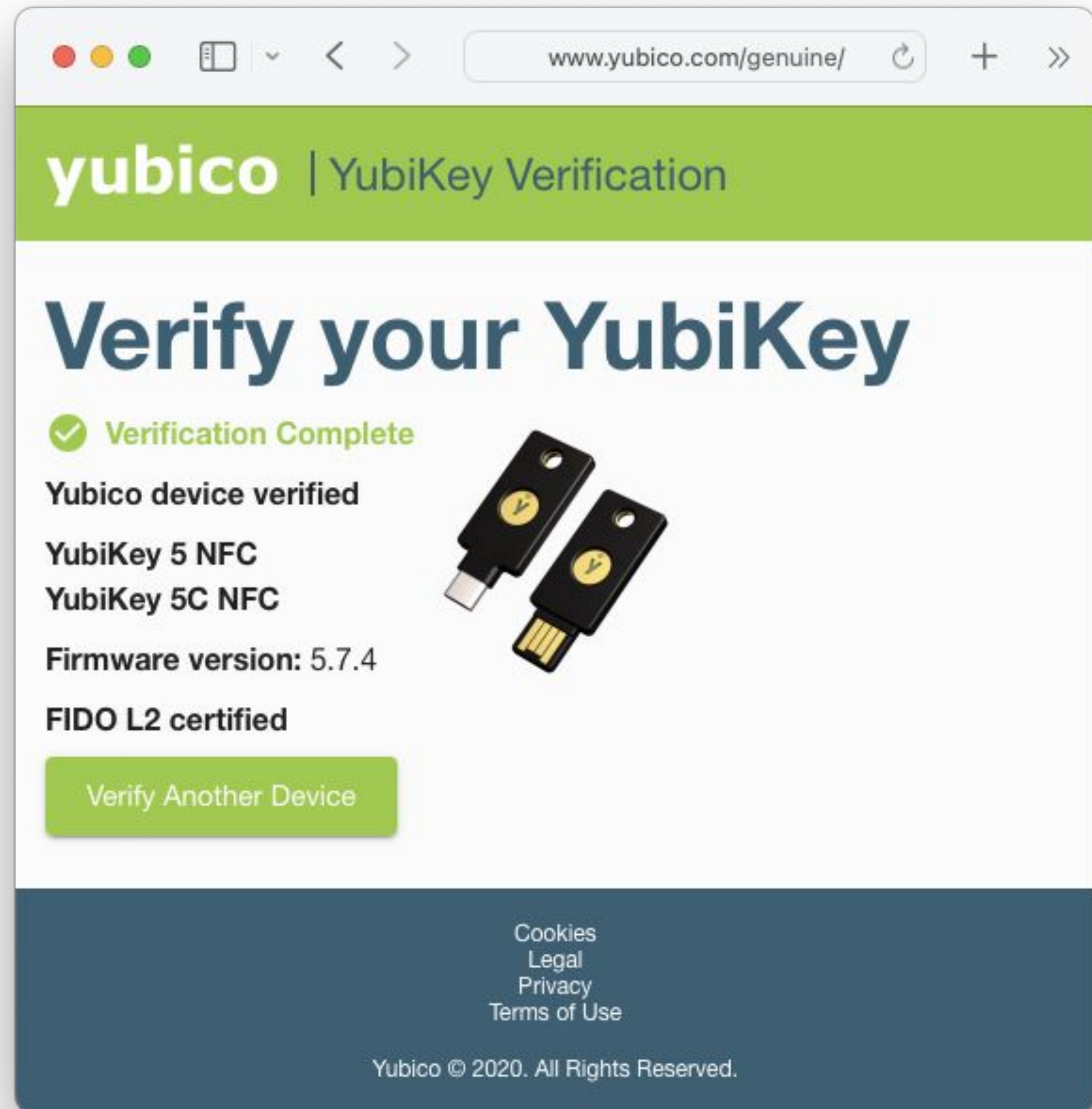
Passwordless sign-in with passkeys



yubico

Attestation

- Proves which make and model of security key generated a credential
- Useful for policies, only allow SSH keys generated on approved hardware



yubico

libfido2

- <https://github.com/Yubico/libfido2>
- Library and command-line tools
- Communicate with a FIDO device over USB or NFC
- Supports both FIDO U2F (CTAP 1) and FIDO2 (CTAP 2) protocols
- Works on Linux, macOS, Windows, OpenBSD, and FreeBSD
- Bindings for [.NET](#), [Go](#), [Perl](#), [Rust](#)
- Also available:
 - <https://github.com/Yubico/python-fido2>

Get Info

\$ fido2-token -L

ioreg://4296874505: vendor=0x1050, product=0x0407 (Yubico YubiKey OTP+FIDO+CCID)

\$ fido2-token -I ioreg://4296874505

proto: 0x02

major: 0x05

minor: 0x07

build: 0x04

caps: 0x05 (wink, cbor, msg)

version strings: U2F_V2, FIDO_2_0, FIDO_2_1_PRE, FIDO_2_1

extension strings: credProtect, hmac-secret, largeBlobKey, credBlob, minPinLength

transport strings: nfc, usb

algorithms: es256 (public-key), eddsa (public-key), es384 (public-key)

aaguid: d7781e5de35346aaafe23ca49f13332a

options: rk, up, noplat, noalwaysUv, credMgmt, authnrCfg, clientPin, largeBlobs, pinUvAuthToken, setMinPINLength, makeCredUvNotRqd

fwversion: 0x50704

maxmsgsiz: 1536

maxcredcntlst: 8

maxcredlen: 128

maxcredblob: 32

maxlargeblob: 4096

maxrpids in minpinlen: 1

remaining rk(s): 98

minpinlen: 4

pin protocols: 2, 1

pin retries: 8

pin change required: false

uv retries: undefined

yubico

Resources

- Developer Program: <https://dev.yubi.co/>
- CTAP 2.1 spec: <https://fidoalliance.org/specs/fido-v2.2-ps-20250228/>
- FIDO dev forum: <https://groups.google.com/a/fidoalliance.org/g/fido-dev>
- libfido2: <https://github.com/Yubico/libfido2>
- FIDO Metadata:
 - <https://fidoalliance.org/metadata/>
 - <https://opotonniee.github.io/fido-mds-explorer/>
- SSH support for security keys:
<https://github.com/openssh/openssh-portable/blob/master/PROTOCOL.u2f>
- OpenSSH Workshop: <https://github.com/YubicoLabs/fido-openssh-workshop/>
- Contact: alan.alvarez@yubico.com | dennis.hills@yubico.com

Questions?